

2Министерство науки и высшего образования Российской Федерации
Муромский институт (филиал)
федерального государственного бюджетного образовательного учреждения высшего образования
«Владимирский государственный университет
Имени Александра Григорьевича и Николая Григорьевича Столетовых»
(МИВлГУ)

Факультет _____ ИТР
Кафедра _____ ПИИ

ЛАБОРАТОРНАЯ РАБОТА №8

по _____ Разработка кроссплатформенных приложений
Тема _____ Разработка слоя доступа к данным

Руководитель

Кульков Я.Ю.

(фамилия, инициалы)

(подпись)

(дата)

Студент ПИИ-124

(группа)

Кочедыков Е.А.

(фамилия, инициалы)

(подпись)

(дата)

Муром 2026

Лабораторная работа № 8

Тема: Разработка слоя доступа к данным

Цель работы: изучить способы разработки слоя доступа к данным

Задание.

Разработать и описать классы доступа к данным (DAO) для выбранной предметной области (по теме КР). Разработка должна включать в себя:

1. Слой моделей данных (Model). Проанализировав задание на разработку, выделить основные сущности. Для каждой из которых создать POJO-классы для представления данных.

2. Интерфейсы слоев доступа к данным (DAO). Интерфейсы должны описывать прототипы методов получения и передачи данных. Данный слой работает с моделями данных, которые записывает в хранилище. Рекомендуется, чтобы методы возвращали не сущности, а Optional<>.

Ход работы:

Листинг 1. dao.impl.BuildingObjectDaoImpl.java

```
package ru.kochedykovdev.laba7.dao.impl;

import ru.kochedykovdev.laba7.dao.BuildingObjectDao;
import ru.kochedykovdev.laba7.model.BuildingObject;
import ru.kochedykovdev.laba7.util.DBHelper;

import java.sql.*;
import java.util.*;

public class BuildingObjectDaoImpl implements BuildingObjectDao {

    private static final String TABLE = DBHelper.SCHEMA + ".building_object";

    @Override
    public void insert(BuildingObject object) throws SQLException {
        String sql = "INSERT INTO " + TABLE + " (name, address, prorab_id) VALUES"
            + "(?, ?, ?) RETURNING id";

        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, object.getName());
        }
    }
}
```

					МИВУ 09.03.04-15-13									
Изм	Лист	№ докум.	Подп.	Дата	Разработка слоя доступа к данным				Лит.		Лист	Листов		
Разраб.		Кочедыков Е.А.										2		26
Провер.		Кульков Я.Ю.							МИВУ ПИН - 124					
Н.контр.														
Утв.														

```

        ps.setString(2, object.getAddress());
        ps.setObject(3, object.getProrabId());
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            object.setId(rs.getLong("id"));
        }
    }
}

@Override
public Optional<BuildingObject> findById(Long id) throws SQLException {
    String sql = "SELECT * FROM " + TABLE + " WHERE id = ?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, id);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            return Optional.of(mapRow(rs));
        }
    }
    return Optional.empty();
}

@Override
public List<BuildingObject> findAll() throws SQLException {
    String sql = "SELECT * FROM " + TABLE;
    List<BuildingObject> list = new ArrayList<>();
    try (Connection conn = DBHelper.getConnection();
        Statement st = conn.createStatement();
        ResultSet rs = st.executeQuery(sql)) {
        while (rs.next()) {
            list.add(mapRow(rs));
        }
    }
    return list;
}

@Override
public void update(BuildingObject object) throws SQLException {
    String sql = "UPDATE " + TABLE + " SET name=?, address=?, prorab_id=? WHERE id=?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, object.getName());
        ps.setString(2, object.getAddress());
    }
}

```

```

        ps.setObject(3, object.getProrabId());
        ps.setLong(4, object.getId());
        ps.executeUpdate();
    }
}

@Override
public void deleteById(Long id) throws SQLException {
    String sql = "DELETE FROM " + TABLE + " WHERE id = ?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, id);
        ps.executeUpdate();
    }
}

private BuildingObject mapRow(ResultSet rs) throws SQLException {
    BuildingObject object = new BuildingObject();
    object.setId(rs.getLong("id"));
    object.setName(rs.getString("name"));
    object.setAddress(rs.getString("address"));
    object.setProrabId((Long) rs.getObject("prorab_id"));
    return object;
}
}

```

Листинг 2. dao.impl. InvoiceDaoImpl.java

```

package ru.kochedykovdev.laba7.dao.impl;

import ru.kochedykovdev.laba7.dao.InvoiceDao;
import ru.kochedykovdev.laba7.model.Invoice;
import ru.kochedykovdev.laba7.util.DBHelper;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class InvoiceDaoImpl implements InvoiceDao {

    private static final String TABLE = DBHelper.SCHEMA + ".invoice";

    @Override
    public void insert(Invoice invoice) throws SQLException {

```

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ 09.03.04-15.13

Лист

4

```
String sql = "INSERT INTO " + TABLE + " (invoice_number, object_id,
issue_date) VALUES (?, ?, ?) RETURNING id";
```

```
try (Connection conn = DBHelper.getConnection();
    PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, invoice.getInvoiceNumber());
    ps.setLong(2, invoice.getObjectId());
    ps.setObject(3, invoice.getIssueDate());
    ResultSet rs = ps.executeQuery();
    if (rs.next()) {
        invoice.setId(rs.getLong("id"));
    }
}
}
```

```
@Override
```

```
public Optional<Invoice> findById(Long id) throws SQLException {
    String sql = "SELECT * FROM " + TABLE + " WHERE id = ?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, id);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            return Optional.of(mapRow(rs));
        }
    }
    return Optional.empty();
}
```

```
@Override
```

```
public List<Invoice> findAll() throws SQLException {
    String sql = "SELECT * FROM " + TABLE;
    List<Invoice> list = new ArrayList<>();
    try (Connection conn = DBHelper.getConnection();
        Statement st = conn.createStatement();
        ResultSet rs = st.executeQuery(sql)) {
        while (rs.next()) {
            list.add(mapRow(rs));
        }
    }
    return list;
}
```

```
@Override
```

```
public void update(Invoice invoice) throws SQLException {
```

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ 09.03.04-15.13

Лист

5

```

        String sql = "UPDATE " + TABLE + " SET invoice_number=?, object_id=?,
issue_date=? WHERE id=?";

        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, invoice.getInvoiceNumber());
            ps.setLong(2, invoice.getObjectId());
            ps.setObject(3, invoice.getIssueDate());
            ps.setLong(4, invoice.getId());
            ps.executeUpdate();
        }
    }

    @Override
    public void deleteById(Long id) throws SQLException {
        String sql = "DELETE FROM " + TABLE + " WHERE id = ?";
        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, id);
            ps.executeUpdate();
        }
    }

    private Invoice mapRow(ResultSet rs) throws SQLException {
        Invoice invoice = new Invoice();
        invoice.setId(rs.getLong("id"));
        invoice.setInvoiceNumber(rs.getString("invoice_number"));
        invoice.setObjectId(rs.getLong("object_id"));
        invoice.setIssueDate(rs.getObject("issue_date",
java.time.LocalDate.class));
        return invoice;
    }
}

```

Листинг 3. dao.impl. InvoiceItemDaoImpl.java

```

package ru.kochedykovdev.laba7.dao.impl;

import ru.kochedykovdev.laba7.dao.InvoiceItemDao;
import ru.kochedykovdev.laba7.model.InvoiceItem;
import ru.kochedykovdev.laba7.util.DBHelper;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

```

					МИВУ 09.03.04-15.13	Лист
						6
Изм.	Лист	№ докум.	Подп.	Дата		

```

import java.util.Optional;

public class InvoiceItemDaoImpl implements InvoiceItemDao {

    private static final String TABLE = DBHelper.SCHEMA + ".invoice_item";

    @Override
    public void insert(InvoiceItem item) throws SQLException {
        String sql = "INSERT INTO " + TABLE + " (invoice_id, material_id, quantity,
price_at_moment) VALUES (?, ?, ?, ?) RETURNING id";
        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, item.getInvoiceId());
            ps.setLong(2, item.getMaterialId());
            ps.setBigDecimal(3, item.getQuantity());
            ps.setBigDecimal(4, item.getPriceAtMoment());
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                item.setId(rs.getLong("id"));
            }
        }
    }

    @Override
    public Optional<InvoiceItem> findById(Long id) throws SQLException {
        String sql = "SELECT * FROM " + TABLE + " WHERE id = ?";
        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, id);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                return Optional.of(mapRow(rs));
            }
        }
        return Optional.empty();
    }

    @Override
    public List<InvoiceItem> findAll() throws SQLException {
        String sql = "SELECT * FROM " + TABLE;
        List<InvoiceItem> list = new ArrayList<>();
        try (Connection conn = DBHelper.getConnection();
            Statement st = conn.createStatement();
            ResultSet rs = st.executeQuery(sql)) {
            while (rs.next()) {

```

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ 09.03.04-15.13

Лист

7

```

        list.add(mapRow(rs));
    }
}

return list;
}

@Override
public void update(InvoiceItem item) throws SQLException {
    String sql = "UPDATE " + TABLE + " SET invoice_id=?, material_id=?,
quantity=?, price_at_moment=? WHERE id=?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, item.getInvoiceId());
        ps.setLong(2, item.getMaterialId());
        ps.setBigDecimal(3, item.getQuantity());
        ps.setBigDecimal(4, item.getPriceAtMoment());
        ps.setLong(5, item.getId());
        ps.executeUpdate();
    }
}

@Override
public void deleteById(Long id) throws SQLException {
    String sql = "DELETE FROM " + TABLE + " WHERE id = ?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, id);
        ps.executeUpdate();
    }
}

private InvoiceItem mapRow(ResultSet rs) throws SQLException {
    InvoiceItem item = new InvoiceItem();
    item.setId(rs.getLong("id"));
    item.setInvoiceId(rs.getLong("invoice_id"));
    item.setMaterialId(rs.getLong("material_id"));
    item.setQuantity(rs.getBigDecimal("quantity"));
    item.setPriceAtMoment(rs.getBigDecimal("price_at_moment"));
    return item;
}
}

```

Листинг 4. dao.impl. MaterialCardDaoImpl.java

					МИВУ 09.03.04-15.13	Лист
						8
Изм.	Лист	№ докум.	Подп.	Дата		

```

package ru.kochedykovdev.laba7.dao.impl;

import ru.kochedykovdev.laba7.dao.MaterialCardDao;
import ru.kochedykovdev.laba7.model.MaterialCard;
import ru.kochedykovdev.laba7.util.DBHelper;

import java.sql.*;
import java.util.*;

public class MaterialCardDaoImpl implements MaterialCardDao {

    private static final String TABLE = DBHelper.SCHEMA + ".material_card";

    @Override
    public void insert(MaterialCard card) throws SQLException {
        String sql = "INSERT INTO " + TABLE + " (name, article, unit,
write_off_rate) VALUES (?, ?, ?, ?) RETURNING id";
        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, card.getName());
            ps.setObject(2, card.getArticle());
            ps.setString(3, card.getUnit());
            ps.setObject(4, card.getWriteOffRate());
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                card.setId(rs.getLong("id"));
            }
        }
    }

    @Override
    public Optional<MaterialCard> findById(Long id) throws SQLException {
        String sql = "SELECT * FROM " + TABLE + " WHERE id = ?";
        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, id);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                return Optional.of(mapRow(rs));
            }
        }
        return Optional.empty();
    }

```

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ 09.03.04-15.13

Лист

9

```

    }

    @Override
    public List<MaterialCard> findAll() throws SQLException {
        String sql = "SELECT * FROM " + TABLE;
        List<MaterialCard> list = new ArrayList<>();
        try (Connection conn = DBHelper.getConnection();
            Statement st = conn.createStatement();
            ResultSet rs = st.executeQuery(sql)) {
            while (rs.next()) {
                list.add(mapRow(rs));
            }
        }
        return list;
    }

    @Override
    public void update(MaterialCard card) throws SQLException {
        String sql = "UPDATE " + TABLE + " SET name=?, article=?, unit=?,
write_off_rate=? WHERE id=?";
        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, card.getName());
            ps.setObject(2, card.getArticle());
            ps.setString(3, card.getUnit());
            ps.setObject(4, card.getWriteOffRate());
            ps.setLong(5, card.getId());
            ps.executeUpdate();
        }
    }

    @Override
    public void deleteById(Long id) throws SQLException {
        String sql = "DELETE FROM " + TABLE + " WHERE id = ?";
        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, id);
            ps.executeUpdate();
        }
    }

    private MaterialCard mapRow(ResultSet rs) throws SQLException {
        MaterialCard card = new MaterialCard();
        card.setId(rs.getLong("id"));
        card.setName(rs.getString("name"));
    }

```

```

        card.setArticle((Integer) rs.getObject("article"));
        card.setUnit(rs.getString("unit"));
        card.setWriteOffRate((Integer) rs.getObject("write_off_rate"));
        return card;
    }
}

```

Листинг 5. dao.impl. MaterialReceiptDaoImpl.java

```

package ru.kochedykovdev.laba7.dao.impl;

import ru.kochedykovdev.laba7.dao.MaterialReceiptDao;
import ru.kochedykovdev.laba7.model.MaterialReceipt;
import ru.kochedykovdev.laba7.util.DBHelper;

import java.sql.*;
import java.util.*;

public class MaterialReceiptDaoImpl implements MaterialReceiptDao {

    private static final String TABLE = DBHelper.SCHEMA + ".material_receipt";

    @Override
    public void insert(MaterialReceipt receipt) throws SQLException {
        String sql = "INSERT INTO " + TABLE + " (material_id, supplier_id, quantity, price, receipt_date) VALUES (?, ?, ?, ?, ?) RETURNING id";
        try (Connection conn = DBHelper.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, receipt.getMaterialId());
            ps.setLong(2, receipt.getSupplierId());
            ps.setBigDecimal(3, receipt.getQuantity());
            ps.setBigDecimal(4, receipt.getPrice());
            ps.setObject(5, receipt.getReceiptDate());
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                receipt.setId(rs.getLong("id"));
            }
        }
    }

    @Override
    public Optional<MaterialReceipt> findById(Long id) throws SQLException {

```

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ 09.03.04-15.13

Лист

11

```

String sql = "SELECT * FROM " + TABLE + " WHERE id = ?";
try (Connection conn = DBHelper.getConnection();
    PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setLong(1, id);
    ResultSet rs = ps.executeQuery();
    if (rs.next()) {
        return Optional.of(mapRow(rs));
    }
}
return Optional.empty();
}

@Override
public List<MaterialReceipt> findAll() throws SQLException {
    String sql = "SELECT * FROM " + TABLE;
    List<MaterialReceipt> list = new ArrayList<>();
    try (Connection conn = DBHelper.getConnection();
        Statement st = conn.createStatement();
        ResultSet rs = st.executeQuery(sql)) {
        while (rs.next()) {
            list.add(mapRow(rs));
        }
    }
    return list;
}

@Override
public void update(MaterialReceipt receipt) throws SQLException {
    String sql = "UPDATE " + TABLE + " SET material_id=?, supplier_id=?,
quantity=?, price=?, receipt_date=? WHERE id=?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, receipt.getMaterialId());
        ps.setLong(2, receipt.getSupplierId());
        ps.setBigDecimal(3, receipt.getQuantity());
        ps.setBigDecimal(4, receipt.getPrice());
        ps.setObject(5, receipt.getReceiptDate());
        ps.setLong(6, receipt.getId());
        ps.executeUpdate();
    }
}

@Override
public void deleteById(Long id) throws SQLException {
    String sql = "DELETE FROM " + TABLE + " WHERE id = ?";

```

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ 09.03.04-15.13

Лист

12

```

        try (Connection conn = DBHelper.getConnection());
            PreparedStatement ps = conn.prepareStatement(sql)) {
                ps.setLong(1, id);
                ps.executeUpdate();
            }
        }

private MaterialReceipt mapRow(ResultSet rs) throws SQLException {
    MaterialReceipt receipt = new MaterialReceipt();
    receipt.setId(rs.getLong("id"));
    receipt.setMaterialId(rs.getLong("material_id"));
    receipt.setSupplierId(rs.getLong("supplier_id"));
    receipt.setQuantity(rs.getBigDecimal("quantity"));
    receipt.setPrice(rs.getBigDecimal("price"));
    receipt.setReceiptDate(rs.getObject("receipt_date",
java.time.LocalDate.class));
    return receipt;
}
}

```

Листинг 6. dao.impl. MaterialReturnDaoImpl.java

```

package ru.kochedykovdev.laba7.dao.impl;

import ru.kochedykovdev.laba7.dao.MaterialReturnDao;
import ru.kochedykovdev.laba7.model.MaterialReturn;
import ru.kochedykovdev.laba7.util.DBHelper;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class MaterialReturnDaoImpl implements MaterialReturnDao {

    private static final String TABLE = DBHelper.SCHEMA + ".material_return";

    @Override
    public void insert(MaterialReturn materialReturn) throws SQLException {
        String sql = "INSERT INTO " + TABLE + " (invoice_id, material_id, quantity,
return_date) VALUES (?, ?, ?, ?) RETURNING id";
        try (Connection conn = DBHelper.getConnection());
            PreparedStatement ps = conn.prepareStatement(sql)) {
                ps.setLong(1, materialReturn.getInvoiceId());

```

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ 09.03.04-15.13

Лист

13

```

        ps.setLong(2, materialReturn.getMaterialId());
        ps.setBigDecimal(3, materialReturn.getQuantity());
        ps.setObject(4, materialReturn.getReturnDate());
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            materialReturn.setId(rs.getLong("id"));
        }
    }
}

@Override
public Optional<MaterialReturn> findById(Long id) throws SQLException {
    String sql = "SELECT * FROM " + TABLE + " WHERE id = ?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, id);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            return Optional.of(mapRow(rs));
        }
    }
    return Optional.empty();
}

@Override
public List<MaterialReturn> findAll() throws SQLException {
    String sql = "SELECT * FROM " + TABLE;
    List<MaterialReturn> list = new ArrayList<>();
    try (Connection conn = DBHelper.getConnection();
        Statement st = conn.createStatement();
        ResultSet rs = st.executeQuery(sql)) {
        while (rs.next()) {
            list.add(mapRow(rs));
        }
    }
    return list;
}

@Override
public void update(MaterialReturn materialReturn) throws SQLException {
    String sql = "UPDATE " + TABLE + " SET invoice_id=?, material_id=?,
quantity=?, return_date=? WHERE id=?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, materialReturn.getInvoiceId());

```

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ 09.03.04-15.13

Лист

14

```

        ps.setLong(2, materialReturn.getMaterialId());
        ps.setBigDecimal(3, materialReturn.getQuantity());
        ps.setObject(4, materialReturn.getReturnDate());
        ps.setLong(5, materialReturn.getId());
        ps.executeUpdate();
    }
}

@Override
public void deleteById(Long id) throws SQLException {
    String sql = "DELETE FROM " + TABLE + " WHERE id = ?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, id);
        ps.executeUpdate();
    }
}

private MaterialReturn mapRow(ResultSet rs) throws SQLException {
    MaterialReturn materialReturn = new MaterialReturn();
    materialReturn.setId(rs.getLong("id"));
    materialReturn.setInvoiceId(rs.getLong("invoice_id"));
    materialReturn.setMaterialId(rs.getLong("material_id"));
    materialReturn.setQuantity(rs.getBigDecimal("quantity"));
    materialReturn.setReturnDate(rs.getObject("return_date",
java.time.LocalDate.class));
    return materialReturn;
}
}

```

Листинг 7. dao.impl. MaterialStockDaoImpl.java

```

package ru.kochedykovdev.laba7.dao.impl;

import ru.kochedykovdev.laba7.dao.MaterialStockDao;
import ru.kochedykovdev.laba7.model.MaterialStock;
import ru.kochedykovdev.laba7.util.DBHelper;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class MaterialStockDaoImpl implements MaterialStockDao {

```

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ 09.03.04-15.13

Лист

15

```

private static final String TABLE = DBHelper.SCHEMA + ".material_stock";

@Override
public void insert(MaterialStock stock) throws SQLException {
    String sql = "INSERT INTO " + TABLE + " (material_id, quantity,
last_updated) VALUES (?, ?, ?) RETURNING id";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, stock.getMaterialId());
        ps.setBigDecimal(2, stock.getQuantity());
        ps.setObject(3, stock.getLastUpdated());
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            stock.setId(rs.getLong("id"));
        }
    }
}

@Override
public Optional<MaterialStock> findById(Long id) throws SQLException {
    String sql = "SELECT * FROM " + TABLE + " WHERE id = ?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, id);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            return Optional.of(mapRow(rs));
        }
    }
    return Optional.empty();
}

@Override
public List<MaterialStock> findAll() throws SQLException {
    String sql = "SELECT * FROM " + TABLE;
    List<MaterialStock> list = new ArrayList<>();
    try (Connection conn = DBHelper.getConnection();
        Statement st = conn.createStatement();
        ResultSet rs = st.executeQuery(sql)) {
        while (rs.next()) {
            list.add(mapRow(rs));
        }
    }
    return list;
}

```

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ 09.03.04-15.13

Лист

16

```

    }

    @Override
    public void update(MaterialStock stock) throws SQLException {
        String sql = "UPDATE " + TABLE + " SET material_id=?, quantity=?,
last_updated=? WHERE id=?";
        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, stock.getMaterialId());
            ps.setBigDecimal(2, stock.getQuantity());
            ps.setObject(3, stock.getLastUpdated());
            ps.setLong(4, stock.getId());
            ps.executeUpdate();
        }
    }

    @Override
    public void deleteById(Long id) throws SQLException {
        String sql = "DELETE FROM " + TABLE + " WHERE id = ?";
        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, id);
            ps.executeUpdate();
        }
    }

    private MaterialStock mapRow(ResultSet rs) throws SQLException {
        MaterialStock stock = new MaterialStock();
        stock.setId(rs.getLong("id"));
        stock.setMaterialId(rs.getLong("material_id"));
        stock.setQuantity(rs.getBigDecimal("quantity"));
        stock.setLastUpdated(rs.getObject("last_updated",
java.time.LocalDateTime.class));
        return stock;
    }
}

```

Листинг 8. dao.impl. ProrabDaoImpl.java

```

package ru.kochedykovdev.laba7.dao.impl;

import ru.kochedykovdev.laba7.dao.ProrabDao;
import ru.kochedykovdev.laba7.model.Prorab;
import ru.kochedykovdev.laba7.util.DBHelper;

```

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ 09.03.04-15.13

Лист

17

```

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class ProrabDaoImpl implements ProrabDao {

    private static final String TABLE = DBHelper.SCHEMA + ".prorab";

    @Override
    public void insert(Prorab prorab) throws SQLException {
        String sql = "INSERT INTO " + TABLE + " (name, phone) VALUES (?, ?)
RETURNING id";

        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, prorab.getName());
            ps.setString(2, prorab.getPhone());
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                prorab.setId(rs.getLong("id"));
            }
        }
    }

    @Override
    public Optional<Prorab> findById(Long id) throws SQLException {
        String sql = "SELECT * FROM " + TABLE + " WHERE id = ?";
        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, id);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                return Optional.of(mapRow(rs));
            }
        }
        return Optional.empty();
    }

    @Override
    public List<Prorab> findAll() throws SQLException {
        String sql = "SELECT * FROM " + TABLE;
        List<Prorab> list = new ArrayList<>();
        try (Connection conn = DBHelper.getConnection();
            Statement st = conn.createStatement();
            ResultSet rs = st.executeQuery(sql)) {
            while (rs.next()) {
                list.add(mapRow(rs));
            }
        }
        return list;
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ 09.03.04-15.13

Лист

18

```

        ResultSet rs = st.executeQuery(sql)) {
            while (rs.next()) {
                list.add(mapRow(rs));
            }
        }
        return list;
    }

    @Override
    public void update(Prorab prorab) throws SQLException {
        String sql = "UPDATE " + TABLE + " SET name=?, phone=? WHERE id=?";
        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, prorab.getName());
            ps.setString(2, prorab.getPhone());
            ps.setLong(3, prorab.getId());
            ps.executeUpdate();
        }
    }

    @Override
    public void deleteById(Long id) throws SQLException {
        String sql = "DELETE FROM " + TABLE + " WHERE id = ?";
        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, id);
            ps.executeUpdate();
        }
    }

    private Prorab mapRow(ResultSet rs) throws SQLException {
        Prorab prorab = new Prorab();
        prorab.setId(rs.getLong("id"));
        prorab.setName(rs.getString("name"));
        prorab.setPhone(rs.getString("phone"));
        return prorab;
    }
}

```

Листинг 9. dao.impl. SupplierDaoImpl.java

```

package ru.kochedykovdev.laba7.dao.impl;

import ru.kochedykovdev.laba7.dao.SupplierDao;

```

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ 09.03.04-15.13

Лист

19

```

import ru.kochedykovdev.laba7.model.Supplier;
import ru.kochedykovdev.laba7.util.DBHelper;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class SupplierDaoImpl implements SupplierDao {

    private static final String TABLE = DBHelper.SCHEMA + ".supplier";

    @Override
    public void insert(Supplier supplier) throws SQLException {
        String sql = "INSERT INTO " + TABLE + " (name, contact_person, phone, email, address) VALUES (?, ?, ?, ?, ?) RETURNING id";
        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, supplier.getName());
            ps.setString(2, supplier.getContactPerson());
            ps.setString(3, supplier.getPhone());
            ps.setString(4, supplier.getEmail());
            ps.setString(5, supplier.getAddress());
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                supplier.setId(rs.getLong("id"));
            }
        }
    }

    @Override
    public Optional<Supplier> findById(Long id) throws SQLException {
        String sql = "SELECT * FROM " + TABLE + " WHERE id = ?";
        try (Connection conn = DBHelper.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, id);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                return Optional.of(mapRow(rs));
            }
        }
        return Optional.empty();
    }

    @Override

```

```

public List<Supplier> findAll() throws SQLException {
    String sql = "SELECT * FROM " + TABLE;
    List<Supplier> list = new ArrayList<>();
    try (Connection conn = DBHelper.getConnection();
        Statement st = conn.createStatement();
        ResultSet rs = st.executeQuery(sql)) {
        while (rs.next()) {
            list.add(mapRow(rs));
        }
    }
    return list;
}

@Override
public void update(Supplier supplier) throws SQLException {
    String sql = "UPDATE " + TABLE + " SET name=?, contact_person=?, phone=?,
email=?, address=? WHERE id=?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, supplier.getName());
        ps.setString(2, supplier.getContactPerson());
        ps.setString(3, supplier.getPhone());
        ps.setString(4, supplier.getEmail());
        ps.setString(5, supplier.getAddress());
        ps.setLong(6, supplier.getId());
        ps.executeUpdate();
    }
}

@Override
public void deleteById(Long id) throws SQLException {
    String sql = "DELETE FROM " + TABLE + " WHERE id = ?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, id);
        ps.executeUpdate();
    }
}

private Supplier mapRow(ResultSet rs) throws SQLException {
    Supplier supplier = new Supplier();
    supplier.setId(rs.getLong("id"));
    supplier.setName(rs.getString("name"));
    supplier.setContactPerson(rs.getString("contact_person"));
    supplier.setPhone(rs.getString("phone"));
}

```

```

        supplier.setEmail(rs.getString("email"));
        supplier.setAddress(rs.getString("address"));
        return supplier;
    }
}

```

Листинг 10. dao.impl. ToolDaoImpl.java

```

package ru.kochedykovdev.laba7.dao.impl;

import ru.kochedykovdev.laba7.dao.ToolDao;
import ru.kochedykovdev.laba7.model.Tool;
import ru.kochedykovdev.laba7.util.DBHelper;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class ToolDaoImpl implements ToolDao {

    private static final String TABLE = DBHelper.SCHEMA + ".tool";

    @Override
    public void insert(Tool tool) throws SQLException {
        String sql = "INSERT INTO " + TABLE + " (name, inventory_number, \"condition\") VALUES (?, ?, ?) RETURNING id";
        try (Connection conn = DBHelper.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, tool.getName());
            ps.setString(2, tool.getInventoryNumber());
            ps.setString(3, tool.getCondition());
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                tool.setId(rs.getLong("id"));
            }
        }
    }

    @Override
    public Optional<Tool> findById(Long id) throws SQLException {
        String sql = "SELECT * FROM " + TABLE + " WHERE id = ?";
        try (Connection conn = DBHelper.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {

```

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ 09.03.04-15.13

Лист

22

```

        ps.setLong(1, id);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            return Optional.of(mapRow(rs));
        }
    }
    return Optional.empty();
}

@Override
public List<Tool> findAll() throws SQLException {
    String sql = "SELECT * FROM " + TABLE;
    List<Tool> list = new ArrayList<>();
    try (Connection conn = DBHelper.getConnection();
        Statement st = conn.createStatement();
        ResultSet rs = st.executeQuery(sql)) {
        while (rs.next()) {
            list.add(mapRow(rs));
        }
    }
    return list;
}

@Override
public void update(Tool tool) throws SQLException {
    String sql = "UPDATE " + TABLE + " SET name=?, inventory_number=?,
\"condition\"=? WHERE id=?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, tool.getName());
        ps.setString(2, tool.getInventoryNumber());
        ps.setString(3, tool.getCondition());
        ps.setLong(4, tool.getId());
        ps.executeUpdate();
    }
}

@Override
public void deleteById(Long id) throws SQLException {
    String sql = "DELETE FROM " + TABLE + " WHERE id = ?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, id);
        ps.executeUpdate();
    }
}

```

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ 09.03.04-15.13

Лист

23

```

    }

    private Tool mapRow(ResultSet rs) throws SQLException {
        Tool tool = new Tool();
        tool.setId(rs.getLong("id"));
        tool.setName(rs.getString("name"));
        tool.setInventoryNumber(rs.getString("inventory_number"));
        tool.setCondition(rs.getString("condition"));
        return tool;
    }
}

```

Листинг 11. dao.impl. ToolIssueDaoImpl.java

```

package ru.kochedykovdev.laba7.dao.impl;

import ru.kochedykovdev.laba7.dao.ToolIssueDao;
import ru.kochedykovdev.laba7.model.ToolIssue;
import ru.kochedykovdev.laba7.util.DBHelper;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class ToolIssueDaoImpl implements ToolIssueDao {

    private static final String TABLE = DBHelper.SCHEMA + ".tool_issue";

    @Override
    public void insert(ToolIssue issue) throws SQLException {
        String sql = "INSERT INTO " + TABLE + " (tool_id, object_id, issued_to, issue_date, return_date, actual_return_date) VALUES (?, ?, ?, ?, ?, ?) RETURNING id";

        try (Connection conn = DBHelper.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, issue.getToolId());
            ps.setLong(2, issue.getObjectId());
            ps.setString(3, issue.getIssuedTo());
            ps.setObject(4, issue.getIssueDate());
            ps.setObject(5, issue.getReturnDate());
            ps.setObject(6, issue.getActualReturnDate());
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {

```

Изм.	Лист	№ докум.	Подп.	Дата

МИВУ 09.03.04-15.13

Лист

24

```

        issue.setId(rs.getLong("id"));
    }
}

@Override
public Optional<ToolIssue> findById(Long id) throws SQLException {
    String sql = "SELECT * FROM " + TABLE + " WHERE id = ?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, id);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            return Optional.of(mapRow(rs));
        }
    }
    return Optional.empty();
}

@Override
public List<ToolIssue> findAll() throws SQLException {
    String sql = "SELECT * FROM " + TABLE;
    List<ToolIssue> list = new ArrayList<>();
    try (Connection conn = DBHelper.getConnection();
        Statement st = conn.createStatement();
        ResultSet rs = st.executeQuery(sql)) {
        while (rs.next()) {
            list.add(mapRow(rs));
        }
    }
    return list;
}

@Override
public void update(ToolIssue issue) throws SQLException {
    String sql = "UPDATE " + TABLE + " SET tool_id=?, object_id=?, issued_to=?,
issue_date=?, return_date=?, actual_return_date=? WHERE id=?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, issue.getToolId());
        ps.setLong(2, issue.getObjectId());
        ps.setString(3, issue.getIssuedTo());
        ps.setObject(4, issue.getIssueDate());
        ps.setObject(5, issue.getReturnDate());
        ps.setObject(6, issue.getActualReturnDate());
    }
}

```

```

        ps.setLong(7, issue.getId());
        ps.executeUpdate();
    }
}

@Override
public void deleteById(Long id) throws SQLException {
    String sql = "DELETE FROM " + TABLE + " WHERE id = ?";
    try (Connection conn = DBHelper.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, id);
        ps.executeUpdate();
    }
}

private ToolIssue mapRow(ResultSet rs) throws SQLException {
    ToolIssue issue = new ToolIssue();
    issue.setId(rs.getLong("id"));
    issue.setToolId(rs.getLong("tool_id"));
    issue.setObjectId(rs.getLong("object_id"));
    issue.setIssuedTo(rs.getString("issued_to"));
    issue.setIssueDate(rs.getObject("issue_date", java.time.LocalDate.class));
    issue.setReturnDate(rs.getObject("return_date",
java.time.LocalDate.class));
    issue.setActualReturnDate(rs.getObject("actual_return_date",
java.time.LocalDate.class));
    return issue;
}
}

```

Вывод: В ходе лабораторной работы я изучил способы разработки слоя доступа к данным

Изм.	Лист	№ докум.	Подп.	Дата